Section 2 of 4 — Process & Data Handling

Career Compass SG · Module 1 Assignment · Group 7

Your time: 3:30 (official range 3–4 min) · The biggest marking weight (25%) sits here

---

Your job in one sentence

Prove we understood this data instead of just loading it. You cover how we cleaned, transformed and explored the data — and you own the best story in the whole talk: the discovery that the dataset was secretly two files stitched together.

Where you sit in the flow

	Section	Time	Official guide
	1 · Business case & objective	2:15	2–3 mins
→ **YOU**	**2 · Process & data handling**	**3:30**	3–4 mins
	3 · Dashboard / app walkthrough	3:00	3–4 mins
	4 · Challenges & learnings	1:15	1–2 mins

Section 1 hands you: "*…how we turned one million messy rows into data we could trust.*" **Your handover to Section 3:** "*That's the data. Now let's see the dashboard we built on top of it.*"

> **Time rule:** if you're running late, cut Part 3 (the two traps). **Part 4 — the June 2023 story — must be told.** Protect it.

Script

Part 1 — Tools and pipeline *(0:00 – 0:40)*

(Slide: the pipeline diagram)

> Everything is **Python and pandas**, in three stages.
>
> The raw file is **273 megabytes — just over a million rows**. We designed everything on a 50,000-row sample first, then scaled up: the full file is read in **six chunks of 200,000 rows**, cleaned, and saved as Parquet. The whole pipeline runs in **15 seconds**, and the dashboard never touches the raw file — only the clean results.

Part 2 — Cleaning: one principle, three examples *(0:40 – 1:30)*

(Slide: cleaning examples — show 3, not all 11)

We wrote eleven cleaning rules. The one principle behind all of them:

If a value is broken, blank the value — but keep the row.

A posting with a broken salary still proves **a job existed** — deleting the row would corrupt our job counts. Three examples:

- One column was **completely empty in all 1.05 million rows** — dropped.
- About **10,000 postings claimed salaries like $180,000 a month** — yearly figures typed into a monthly field. We blanked the salary, kept the posting.
- The most extreme 1% of salaries were **capped, not deleted**, so a few executive pay packages can't drag every average upward.

After all the cleaning, **99.6% of rows survive**, with salary disclosed on 99%.

Part 3 — Two traps from exploration *(1:30 – 2:10)*

(Slide: the two traps)

Exploring the data caught two traps worth sharing.

The duplicate check lied. pandas reported 289 duplicate job IDs — all 289 turned out to be **empty cells compared with each other**. Real duplicates: **zero**.

And one job can belong to up to three categories — 1.69 on average. Count rows after splitting by category and you double-count jobs. We made this mistake ourselves: for an hour our own front page said **1.66 million postings instead of 1.04 million** — and it looked completely believable. We fixed it by giving every posting an ID that all our market-level numbers must de-duplicate on.

Part 4 — The discovery that changed everything *(2:10 – 3:20)*

(Slide: the two-line engagement chart — speak slowly here)

Now the big one.

As a routine check we plotted views and applications **over time**. We weren't looking for a problem. Look at **June 2023**: average views per posting **fall from about 111 to about 5**, and postings with **zero applications jump from 18% to 79%**.

Did Singaporeans stop applying for jobs? Of course not. **This dataset is two separate downloads stitched together.** Postings after June 2023 were captured the moment they went up — their counters **never had time to grow**. They're frozen at zero.

So we made a firm rule: **job counts and salaries use all 1.04 million rows. Competition uses only postings up to June 2023** — about 200,000 — and every page that shows competition says so on screen.

If we had missed this, every career track would have looked like it had **no competition at all**, and our recommender would have sent people into the most crowded markets in Singapore. **That is the difference between a dashboard and a wrong dashboard.**

That's the data. Now — the dashboard we built on top of it.

Numbers you must know cold

Figure	Value
Raw file	**273 MB · 1,048,585 rows**, read in 6 × 200k chunks
Pipeline runtime	**~15 seconds**
Rows kept	**1,044,597 (99.6%)**
Impossible salaries blanked	~10,000
Salary rows capped (top/bottom 1%)	19,647
Real duplicate job IDs	**0** (the "289" were blanks)
Categories per job	**1.69 average**, max 3
The break	**views 111 → 5 · zero-application 18% → 79% · at June 2023**
Competition window	postings up to **30 June 2023** (~203,702)

Questions you own

"Why cap outliers instead of deleting them?"

> Deleting the row also deletes the job from our demand counts. Capping keeps the counts honest while stopping extreme values from distorting averages.

"How do you know the June 2023 break is a data problem and not real?"

> Only the engagement counters break. Posting volume, salaries and categories all continue smoothly across that date. A real market collapse would show up everywhere.

"Isn't 200,000 postings too few for competition?"

> It's still two hundred thousand postings — plenty. The alternative was using numbers we *know* are frozen at zero. A smaller honest number beats a bigger wrong one.

"Why drop rows with missing values instead of filling them in?"

> We checked first: the gaps were all on the *same* rows — completely empty lines with no ID, no title, no dates. There was nothing to fill in from.

"Why Parquet / why not a database?"

> Parquet is compressed and ~20× faster to read than re-parsing the CSV. And nothing here needs a query engine — pandas processes the whole file in 15 seconds.

Rehearsal checklist

- [ ] Timed at **3:30 or under** — and you know Part 3 is the cut if late
- [ ] The June 2023 story delivered from memory, without reading
- [ ] One-sentence explanation ready: the counters are *frozen*, not wrong
- [ ] Comfortable owning the 1.66-million bug — it reads as competence
- [ ] Handover line practised